Name:Kavitha
Reg no:192565041
Course code:DSA0106

UNIT I: Principles of Object Oriented Programming & Control Structures

Section A: Debugging Questions

Question 1: Operator Precedence Trap with Bitwise Operations & Implicit Promotion

Topic: Operators in C++, Operator Precedence, Implicit Conversion

Step 1 – Analyze the problem

Expected: print "Valid Odd" only when `num` is odd **and** greater than 10, then count 3, 2, 1, 0 with an unsigned counter. Actual: the odd test is only correct by coincidence, and the countdown loop never terminates unless the `break` patch is present.

Step 2 – Defects found

Line	Classification	Root cause (C++ rule)
7	Logic Defect (precedence trap)	`==` has higher precedence than bitwise `&`, so `num & 1 == 1 && num > 10` is parsed as `(num & (1 == 1)) && (num > 10)`. It works for mask 1 only because `1 == 1` happens to evaluate to 1.
14	Logic Defect (infinite loop)	`i >= 0` is always true for an unsigned type. When `i` is 0, `--i` wraps around (modulo 256) to 255, so the condition never fails. The `break` is only a patch.

Task 1 – Precedence conflict on Line 7

Precedence (high to low) for the operators involved: relational (>) → equality (==) → bitwise AND (&) → logical AND (&&). The compiler therefore evaluates, for `num = 15`:

- `num > 10` → `15 > 10` → true (1)
- `1 == 1` → true (1) — this is evaluated **before** the &
- `num & 1` → `15 & 1` → 1 (num is promoted from `unsigned char` to `int` first)
- `1 && 1` → true

The intended grouping was `(num & 1) == 1`. Because the compiler instead computes `num & (1 == 1)`, the code is only right for the single mask value 1. For any other mask it fails: e.g. `num & 2 == 2` becomes `num & 1`, so for `num = 2` it gives `2 & 1 = 0` (false) although bit 1 is set. Always parenthesize bitwise sub-expressions.

Task 2 – The for loop on Line 14

`unsigned char` can only hold 0–255, so `i >= 0` can never be false (it is promoted to `int` for the comparison, but its value is never negative). After printing 0, `--i` does not produce −1; unsigned arithmetic wraps to 255, the test passes again, and the loop counts 255, 254, … forever. Without the

`if (i == 0) break;` patch the program therefore loops infinitely. The correct fix is a signed counter (`int i`).

Task 3 – Corrected code

Corrected code

```
#include <iostream>

int main() {
    unsigned char num = 15;

    // Line 7 (fixed): parenthesise the bitwise test
    if (((num & 1) == 1) && (num > 10)) {
        std::cout << "Valid Odd Number: " << (int)num << std::endl;
    } else {
        std::cout << "Condition Failed for: " << (int)num << std::endl;
    }

    // Line 14 (fixed): signed counter, no wrap-around
    for (int i = 3; i >= 0; --i) {
        std::cout << i << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Task 4 – Dry run and output

Step	Evaluation	Result
`(num & 1) == 1`	15 = 0000 1111; AND 0000 0001 = 1; `1 == 1`	true
`num > 10`	15 > 10	true
&&	true AND true	prints "Valid Odd Number: 15"
for: i = 3, 2, 1, 0	each test `i >= 0` is true; prints i and a space	"3 2 1 0 "
for: i = −1	`-1 >= 0` is false	loop exits

Console output

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution & Variable Shadowing Conflicts

Topic: Variables, Scope Resolution Operator (::), Constants

Step 1 – Analyze the problem

The program should show the 5% local rate, the 18% global total and the local total. A syntax error blocks compilation, and once that is fixed the inner calculation silently uses the wrong rate.

Step 2 – Defects found

Line	Classification	Root cause (C++ rule)
11	Compilation Error (syntax)	`:taxRate` uses a single colon. The scope resolution operator is the single token `::`; a lone `:` is not valid in an expression.
16	Logic Defect (semantic)	`::taxRate` always names the global variable (0.18), not the outer local (0.05). The code compiles but computes 1180 instead of the intended 1050.
14	Design flaw (shadowing)	`int taxRate = 12;` hides the outer local and is also the wrong type/meaning (an integer percentage versus fractional rates).

Task 1 – Syntax error on Line 11

`itemPrice * :taxRate` must be `itemPrice * ::taxRate`. The unary `::` (empty left-hand side) tells the compiler to look up the name in the global namespace.

Task 2 – What `::taxRate` resolves to on Line 16

It resolves to the global `taxRate` (0.18), so `totalWithLocalTax` becomes 1000 + 1000 × 0.18 = 1180. The unary `::` operator can **only** reach namespace-scope (global) names; it cannot access a variable of an enclosing function or block scope.

- Name lookup starts in the innermost block and moves outward; the first declaration found wins.
- An inner declaration with the same name **shadows** (hides) every outer one for the rest of that block.
- Once `int taxRate` is declared in the inner block, the outer local `taxRate` (0.05) cannot be named at all from there. Workarounds: give each variable a unique name, or bind a reference/copy to the outer variable before the inner block begins.

Task 3 – Corrected code

Each rate gets a distinct name, so no shadowing remains and `::` is used only for the real global.

Corrected code

```
#include <iostream>

double globalTaxRate = 0.18;       // Global standard tax rate (18%)

int main() {
    double itemPrice    = 1000.0;
    double localTaxRate = 0.05;    // Local subsidised rate (5%)

    std::cout << "Local Rate: " << localTaxRate << std::endl;

    double totalWithGlobalTax = itemPrice + (itemPrice * ::globalTaxRate);

    {
        int corporateTaxRate = 12; // Inner-block variable, unique name
        double totalWithLocalTax = itemPrice + (itemPrice * localTaxRate);
        std::cout << "Inner total: " << totalWithLocalTax << std::endl;
        std::cout << "Corporate rate (%): " << corporateTaxRate << std::endl;
    }

    std::cout << "Final Global Calculation: " << totalWithGlobalTax << std::endl;
    return 0;
}
```

Task 4 – Dry run and expected output

Statement	Calculation	Value
print `localTaxRate`	—	0.05
`totalWithGlobalTax`	1000 + 1000 × 0.18	1180
`totalWithLocalTax` (inner block)	1000 + 1000 × 0.05	1050
print `corporateTaxRate`	—	12

Console output

```
Local Rate: 0.05
Inner total: 1050
Corporate rate (%): 12
Final Global Calculation: 1180
```

(The "Corporate rate" line was added only to show the inner variable is still usable; with the original three print statements, the last two lines are the same.)

Question 3: Switch-Case Variable Initialization & Illegal Case Types

Topic: Control Structures (switch-case), Tokens, Data Types

Step 1 – Analyze the problem

The switch should compute `tokenValue` per token category. It fails to compile because of declarations inside case labels and a floating-point case label, and even after that, `case 2` falls through into `case 3`.

Step 2 – Defects found

Line	Classification	Root cause (C++ rule)
`case 2:` / `int multiplier = 5;`	Compilation Error	"Jump to case label crosses initialization": the labels after the declaration can be jumped to, bypassing the initializer while the variable is still in scope.
`case 4.5:`	Compilation Error	A case label must be an integral or enumeration constant expression; a `double` literal is not allowed.
end of `case 2`	Logic Defect	Missing `break` makes execution fall through into `case 3`.

Task 1 – Cross-initialization error

The whole `switch` body is a single block scope, and each `case` label is just a jump target into it. `int multiplier = 5;` is in scope from its declaration to the closing brace, so `case 3:` and `default:` can jump past the initializer and land where `multiplier` is visible but never initialized. C++ forbids this. The same applies to `int offset = 10;` in `case 1` (the compiler reports it at the later labels). The fix is to enclose each case body that declares variables in its own braces `{ ... }`, so the variable goes out of scope before the next label.

Task 2 – Why `case 4.5:` is rejected

The rule: each case label must be a constant expression of integral or enumeration type, converted to the promoted type of the switch condition (narrowing conversions such as double → int are not allowed). Legal types for both the switch expression and case labels: `int`, `short`, `long`, `long long`, `char`, `bool`, their signed/unsigned variants, `wchar_t`/`char16_t`/`char32_t`, and enumerations (including `enum class`). A class type is accepted only if it has a single non-explicit conversion to an integral/enum type. `float`, `double`, `std::string`, pointers and arrays are **not** allowed. Labels must also be unique.

Task 3 – Logic defect when tokenType == 2

Without `break`, execution continues into `case 3:`. `tokenValue` becomes 20 × 5 = 100, then `tokenValue += 100.0f` makes it **200** instead of 100, and "Calculated Value: 200" is printed, which is wrong.

Task 4 – Corrected code

Corrected code

```
#include <iostream>

int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }
        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;                        // stops fall-through into case 3
        }
        case 3:
            tokenValue += 100.0f;
            break;
        case 4:                           // integral label (was 4.5)
            std::cout << "Float token detected" << std::endl;
            break;
        default:
            std::cout << "Unknown token" << std::endl;
    }

    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

Dry run: `tokenType = 2` → jumps to `case 2` → `multiplier = 5` → `tokenValue = 20.0 × 5 = 100.0` → `break` leaves the switch → value printed.

Console output

```
Calculated Value: 100
```

Question 4: Reference Variable Initialization & Constant Binding Violations

Topic: Reference Variables, Constants, Implicit Conversions

Step 1 – Analyze the problem

Four reference operations each break a different C++ reference rule. (Line numbers follow the question: 7, 11, 14 and 18; the statements are `int &refA;`, `int &refLiteral = 100;`, `int &refInt = pi;` and `constRef = constRef + 10;`.)

Step 2 – Defects found and Task 1 – rules violated

Line	Classification	Rule violated
7	Compilation Error	A reference must be initialized at the point of declaration. `int &refA;` has no referent. (`refA = a;` afterwards would be an assignment through the reference, never a binding, because references cannot be re-seated.)
11	Compilation Error	A non-const lvalue reference cannot bind to an rvalue. The literal `100` has no address and no name.
14	Compilation Error	`int&` cannot bind to a `double`: the conversion double → int creates a temporary (an rvalue), and a non-const lvalue reference cannot bind to a temporary. It would also not be an alias of `pi`.
18	Compilation Error	`constRef` is a reference to const; assigning through it ("assignment of read-only reference") is illegal.

Task 2 – Lifetime extension of temporaries

A temporary normally dies at the end of the full expression that created it. When a temporary is bound **directly** to a `const` lvalue reference (or an rvalue reference), its lifetime is extended to that of the reference, i.e. to the end of the reference's scope. So `const int &constRef = 50;` creates a hidden `int` temporary holding 50 that lives as long as `constRef`, and `const int &r = pi;` (double → int) creates a temporary `int` of value 3 that stays valid. The const is required because it guarantees the temporary cannot be modified, which would be pointless as no one else can see it. Limits: the extension applies only to the temporary bound directly, and not when the reference is returned from a function.

Task 3 – Corrected code

Corrected code

```
#include <iostream>

int main() {
    int a = 25;

    int &refA = a;                      // bound at declaration

    const int &refLiteral = 100;        // const ref may bind to an rvalue

    double pi = 3.14159;
    double &refPi = pi;                 // same type -> valid alias

    const int &constRef = 50;           // read-only view of a temporary
    int updated = constRef + 10;        // compute into a normal variable

    refA = refA + 5;                    // modify a through its alias

    std::cout << a << " " << refA << std::endl;
    std::cout << refLiteral << " " << refPi << std::endl;
    std::cout << constRef << " " << updated << std::endl;
    return 0;
}
```

Task 4 – Dry run and expected output

Statement	State after execution
`int a = 25; int &refA = a;`	refA is an alias of a (25)
`const int &refLiteral = 100;`	temporary 100 lives as long as refLiteral
`double &refPi = pi;`	alias of pi (3.14159)
`const int &constRef = 50; int updated = constRef + 10;`	constRef = 50, updated = 60
`refA = refA + 5;`	a = 30 (refA = 30)

Console output

```
30 30
100 3.14159
50 60
```

If the original single print statement (a and `refA`) is kept, the line is "30 30" here because a was changed through `refA`; without the `refA = refA + 5;` statement it would print "25 25".

Question 5: Dynamic Array Allocation & Stray Loop Semicolon

Topic: Memory Operators (new, delete), Loops, Off-by-One Boundaries

Step 1 – Analyze the problem

The program must store 10, 20, 30, 40, 50 on the heap, sum them and free the memory. A stray semicolon breaks the loop and the deallocation form is wrong.

Step 2 – Defects found

Line	Classification	Root cause (C++ rule)
10	Compilation Error (stray `;`)	The semicolon is an empty loop body. The `{ ... }` block is a separate compound statement and `i` is out of scope there ("'i' was not declared in this scope").
21	Undefined Behavior	`new int[size]` must be released with `delete[]`, not `delete`.

Task 1 – The semicolon on Line 10

`for (int i = 0; i < size; ++i);` ends the statement, so the loop body is empty: the loop just counts `i` from 0 to 5 and does nothing. The following `{ readings[i] = (i + 1) * 10; }` is an ordinary block that runs **once**, after the loop. Since `i` is declared in the for-header, its scope ends with the for statement, so the compiler rejects `readings[i]` as undeclared. If `i` had been declared before the loop, it would equal 5 at that point, so the block would write `readings[5]`, one element past the end (off-by-one heap buffer overflow, memory corruption), and elements 0–4 would stay uninitialized, giving a garbage total.

Task 2 – `delete` versus `delete[]` (Line 21)

`delete p` destroys and frees **one** object allocated with plain `new`. `delete[] p` destroys **every element** of an array allocated with `new[]` (calling each destructor) and releases the whole block, using the element count the runtime stored when the array was created. Using `delete` on memory from `new[]` is undefined behavior: it may leak, corrupt the heap or crash, even for `int` where no destructors exist.

Task 3 – Corrected program

Corrected code

```
#include <iostream>

int main() {
    int size = 5;

    int *readings = new int[size];

    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }
    std::cout << "Total: " << total << std::endl;

    delete[] readings;
    readings = nullptr;

    return 0;
}
```

Task 4 – Trace and final output

i	readings[i] = (i + 1) × 10	total += readings[i]	total
0	10	0 + 10	10
1	20	10 + 20	30
2	30	30 + 30	60
3	40	60 + 40	100
4	50	100 + 50	150

Console output

```
Total: 150
```

Section B: Pseudo-Code Writing Questions

Question 1: Procedural-to-OOP Paradigm Transformation

Topic: OOP Paradigm, Encapsulation, Data Hiding, Abstraction

Design Overview

- **Encapsulation / data hiding:** all consumer data is PRIVATE; the outside world uses only the PUBLIC methods.
- **Abstraction:** the driver calls `Initialize`, `CalculateConsumption`, `ComputeBill`, `DisplayBill` without knowing the tariff logic.
- **Validation in one place:** `Initialize` rejects negative readings and `currentReading < previousReading`; the other methods do nothing on invalid data.
- **Tiered tariff** with nested IF: 100 units at $1.50, next 200 at $2.50, above 300 at $4.00.
- **Assumptions:** the 10% environmental surcharge applies to the entire bill only when consumption exceeds 300 units. Two helper attributes were added (`surcharge`, `isValid`).

Task 1 – Structured pseudo-code

Structured pseudo-code

```
CLASS WaterConsumer
  PRIVATE:
    STRING  consumerID
    REAL    previousReading
    REAL    currentReading
    REAL    unitsConsumed
    REAL    totalBill
    REAL    surcharge
    BOOLEAN isValid

  PUBLIC:
    CONSTRUCTOR WaterConsumer()
      consumerID      <- ""
      previousReading <- 0.0
      currentReading  <- 0.0
      unitsConsumed   <- 0.0
      totalBill       <- 0.0
      surcharge       <- 0.0
      isValid         <- FALSE
    ENDCONSTRUCTOR

    METHOD Initialize(STRING id, REAL prev, REAL curr) RETURNS BOOLEAN
      IF prev < 0 OR curr < 0 THEN
        DISPLAY "ERROR: Readings must be non-negative"
        isValid <- FALSE
        RETURN FALSE
      ENDIF
      IF curr < prev THEN
        DISPLAY "ERROR: Current reading is below previous reading"
        isValid <- FALSE
        RETURN FALSE
      ENDIF
      consumerID      <- id
      previousReading <- prev
      currentReading  <- curr
      isValid         <- TRUE
      RETURN TRUE
    ENDMETHOD
```

```
    METHOD CalculateConsumption()
      IF isValid = TRUE THEN
        unitsConsumed <- currentReading - previousReading
      ENDIF
    ENDMETHOD

    METHOD ComputeBill()
      REAL tier1Units, tier2Units, tier3Units, baseBill
      IF isValid = FALSE THEN
        RETURN
      ENDIF
      tier1Units <- 0.0
      tier2Units <- 0.0
      tier3Units <- 0.0
      IF unitsConsumed <= 100 THEN
        tier1Units <- unitsConsumed
      ELSE
        tier1Units <- 100
        IF unitsConsumed <= 300 THEN
          tier2Units <- unitsConsumed - 100
        ELSE
          tier2Units <- 200
          tier3Units <- unitsConsumed - 300
        ENDIF
      ENDIF
      baseBill <- tier1Units * 1.50 + tier2Units * 2.50
                  + tier3Units * 4.00
      IF unitsConsumed > 300 THEN
        surcharge <- baseBill * 0.10
      ELSE
        surcharge <- 0.0
      ENDIF
      totalBill <- baseBill + surcharge
    ENDMETHOD

    METHOD DisplayBill()
      IF isValid = FALSE THEN
        DISPLAY "No valid bill for this consumer"
        RETURN
      ENDIF
      DISPLAY "Consumer ID    : ", consumerID
      DISPLAY "Previous / Curr: ", previousReading, " / ", currentReading
      DISPLAY "Units consumed : ", unitsConsumed
      DISPLAY "Surcharge      : $", surcharge
      DISPLAY "TOTAL PAYABLE  : $", totalBill
    ENDMETHOD
ENDCLASS

BEGIN Main
  WaterConsumer consumer
  BOOLEAN ok
  ok <- consumer.Initialize("WC-1001", 1450.0, 1820.0)
  IF ok = TRUE THEN
    consumer.CalculateConsumption()
    consumer.ComputeBill()
    consumer.DisplayBill()
  ELSE
    DISPLAY "Billing aborted: invalid meter data"
  ENDIF
END
```

Task 2 – Trace table (previousReading = 1450.0, currentReading = 1820.0)

Step	Operation	Working	Result
1	Initialize	1450 ≥ 0, 1820 ≥ 0, 1820 ≥ 1450	TRUE (valid)
2	CalculateConsumption	1820.0 − 1450.0	unitsConsumed = 370.0
3	Tier 1 (first 100)	370 > 100 → tier1Units = 100; 100 × $1.50	$150.00
4	Tier 2 (101–300)	370 > 300 → tier2Units = 200; 200 × $2.50	$500.00
5	Tier 3 (above 300)	tier3Units = 370 − 300 = 70; 70 × $4.00	$280.00
6	baseBill	150.00 + 500.00 + 280.00	$930.00
7	Surcharge	370 > 300 → 10% × 930.00	$93.00
8	totalBill	930.00 + 93.00	$1,023.00

Question 2: Multi-Tier Tax & Precision Cast Engine

Topic: Control Structures (Selection), Data Types, Type Casting

Design Overview

- **Clamp deductions** to their statutory caps ($150,000 and $25,000) using IF/ELSE, then clamp taxable income at zero.
- **Progressive slabs** are applied cumulatively with nested IF: tax at the lower slab boundaries is precomputed ($12,500 at $500,000; $112,500 at $1,000,000).
- **Cess:** 4% is added on the total raw tax.
- **Type cast:** the REAL result is converted to INTEGER with `CAST_TO_INTEGER(tax + 0.5)`, which rounds to the nearest whole unit (the cast truncates toward zero).

Task 1 – Structured pseudo-code

Structured pseudo-code

```
BEGIN TaxCalculationEngine
  CONSTANT REAL CAP_80C         <- 150000.0
  CONSTANT REAL CAP_HEALTH      <- 25000.0
  CONSTANT REAL SLAB1_LIMIT     <- 250000.0
  CONSTANT REAL SLAB2_LIMIT     <- 500000.0
  CONSTANT REAL SLAB3_LIMIT     <- 1000000.0
  CONSTANT REAL TAX_AT_SLAB2    <- 12500.0     // 5% of 250,000
  CONSTANT REAL TAX_AT_SLAB3    <- 112500.0    // 12,500 + 20% of 500,000
  CONSTANT REAL CESS_RATE       <- 0.04

  REAL grossAnnualSalary, deduction80C, deductionHealthInsurance
  REAL eligible80C, eligibleHealth, totalDeductions
  REAL taxableIncome, rawTax, cess, totalTax
  INTEGER finalTax

  READ grossAnnualSalary, deduction80C, deductionHealthInsurance

  // Step 1: apply statutory caps
  IF deduction80C > CAP_80C THEN
    eligible80C <- CAP_80C
  ELSE
    eligible80C <- deduction80C
  ENDIF
  IF deductionHealthInsurance > CAP_HEALTH THEN
    eligibleHealth <- CAP_HEALTH
  ELSE
    eligibleHealth <- deductionHealthInsurance
  ENDIF

  // Step 2: taxable income, clamped to zero
  totalDeductions <- eligible80C + eligibleHealth
  taxableIncome   <- grossAnnualSalary - totalDeductions
  IF taxableIncome < 0 THEN
    taxableIncome <- 0.0
  ENDIF

  // Step 3: progressive brackets
  IF taxableIncome <= SLAB1_LIMIT THEN
    rawTax <- 0.0
  ELSE
    IF taxableIncome <= SLAB2_LIMIT THEN
      rawTax <- 0.05 * (taxableIncome - SLAB1_LIMIT)
```

```
    ELSE
      IF taxableIncome <= SLAB3_LIMIT THEN
        rawTax <- TAX_AT_SLAB2 + 0.20 * (taxableIncome - SLAB2_LIMIT)
      ELSE
        rawTax <- TAX_AT_SLAB3 + 0.30 * (taxableIncome - SLAB3_LIMIT)
      ENDIF
    ENDIF
  ENDIF

  // Step 4: cess, then Step 5: explicit REAL -> INTEGER cast
  cess     <- rawTax * CESS_RATE
  totalTax <- rawTax + cess
  finalTax <- CAST_TO_INTEGER(totalTax + 0.5)

  DISPLAY "Taxable income : ", taxableIncome
  DISPLAY "Raw tax        : ", rawTax
  DISPLAY "Cess (4%)      : ", cess
  DISPLAY "Final tax      : $", finalTax
END
```

Task 2 – Trace table (gross = $1,200,000; 80C = $180,000; health = $30,000)

Step	Operation	Working	Result
1	Cap 80C	180,000 > 150,000 → capped	eligible80C = 150,000
2	Cap health	30,000 > 25,000 → capped	eligibleHealth = 25,000
3	Total deductions	150,000 + 25,000	175,000
4	Taxable income	1,200,000 − 175,000 (≥ 0, no clamp)	1,025,000
5	Slab selection	1,025,000 > 1,000,000 → top slab	rawTax uses TAX_AT_SLAB3
6	Slab 3 tax	112,500 + 0.30 × (1,025,000 − 1,000,000) = 112,500 + 7,500	120,000.00
7	Cess 4%	0.04 × 120,000	4,800.00
8	totalTax	120,000 + 4,800	124,800.00
9	Cast	CAST_TO_INTEGER(124,800.00 + 0.5) = CAST(124,800.5)	124,800

Check of the 112,500 figure: 5% × 250,000 = 12,500 (slab 2) + 20% × 500,000 = 100,000 (slab 3) = 112,500.

Question 3: Bitmask Validation & Sensor Health Telemetry

Topic: Operators in C++ (Bitwise &, |, ^, ~), Tokens, Constants

Design Overview

- **Named hexadecimal constants** for every flag, then two composite masks built with OR.
- **One test for readiness:** clear the non-blocking radio bit with `AND (NOT RADIO_LINK)` and compare with the mandatory mask (0x0F). This works because the inhibitor bits must be 0 and the mandatory bits must be 1.
- **Inhibitor diagnostics:** `status AND INHIBITOR_MASK` isolates active faults; each bit is then identified with its own mask test (no loop).
- **Missing sensors:** `(status AND MANDATORY_MASK) XOR MANDATORY_MASK` gives a 1 for every mandatory bit that is 0.

Task 1 – Hexadecimal bitmask constants

Constant	Hex	Binary	Meaning
GYRO_LOCKED	0x01	0000 0001	Bit 0 – Gyroscope locked
ACCEL_CALIBRATED	0x02	0000 0010	Bit 1 – Accelerometer calibrated
BARO_ONLINE	0x04	0000 0100	Bit 2 – Barometric altimeter online
GPS_FIX	0x08	0000 1000	Bit 3 – GPS fix acquired
LOW_BATTERY	0x10	0001 0000	Bit 4 – Low battery warning
OVER_TEMP	0x20	0010 0000	Bit 5 – Over-temperature warning
RADIO_LINK	0x40	0100 0000	Bit 6 – Radio link (non-blocking)
ACTUATOR_FATAL	0x80	1000 0000	Bit 7 – Fatal actuator failure
MANDATORY_MASK	0x0F	0000 1111	GYRO OR ACCEL OR BARO OR GPS
INHIBITOR_MASK	0xB0	1011 0000	LOW_BATTERY OR OVER_TEMP OR ACTUATOR_FATAL

Task 2 – Structured pseudo-code

Structured pseudo-code

```
FUNCTION CheckFlightReadiness(BYTE statusRegister) RETURNS BOOLEAN
  BYTE relevantBits, activeFaults, missingSensors

  // radio link is non-blocking: mask it out
  relevantBits <- statusRegister AND (NOT RADIO_LINK)
  IF relevantBits = MANDATORY_MASK THEN
    DISPLAY "FLIGHT READY"
    RETURN TRUE
  ENDIF

  // inhibitors: any of bits 4, 5, 7 set
  activeFaults <- statusRegister AND INHIBITOR_MASK
  IF activeFaults <> 0x00 THEN
    IF (activeFaults AND LOW_BATTERY) <> 0 THEN
      DISPLAY "FAULT: Bit 4 - Low Battery Warning"
    ENDIF
```

```
    IF (activeFaults AND OVER_TEMP) <> 0 THEN
      DISPLAY "FAULT: Bit 5 - Over-Temperature Warning"
    ENDIF
    IF (activeFaults AND ACTUATOR_FATAL) <> 0 THEN
      DISPLAY "FAULT: Bit 7 - Fatal Actuator Hardware Failure"
    ENDIF
  ENDIF

  // mandatory sensors: XOR shows which required bits are 0
  missingSensors <- (statusRegister AND MANDATORY_MASK) XOR MANDATORY_MASK
  IF missingSensors <> 0x00 THEN
    IF (missingSensors AND GYRO_LOCKED) <> 0 THEN
      DISPLAY "MISSING: Bit 0 - Gyroscope not locked"
    ENDIF
    IF (missingSensors AND ACCEL_CALIBRATED) <> 0 THEN
      DISPLAY "MISSING: Bit 1 - Accelerometer not calibrated"
    ENDIF
    IF (missingSensors AND BARO_ONLINE) <> 0 THEN
      DISPLAY "MISSING: Bit 2 - Barometric altimeter offline"
    ENDIF
    IF (missingSensors AND GPS_FIX) <> 0 THEN
      DISPLAY "MISSING: Bit 3 - No GPS fix"
    ENDIF
  ENDIF

  DISPLAY "FLIGHT NOT READY"
  RETURN FALSE
ENDFUNCTION
```

Task 3 – Traces

Operation	Test byte 0x4F (0100 1111)	Test byte 0x3B (0011 1011)
`NOT RADIO_LINK`	0xBF (1011 1111)	0xBF (1011 1111)
relevantBits = status AND 0xBF	0000 1111 = 0x0F	0011 1011 = 0x3B
relevantBits = MANDATORY_MASK (0x0F)?	Yes → "FLIGHT READY", return TRUE	No → run diagnostics
activeFaults = status AND 0xB0	0x00 (not reached)	0011 0000 = 0x30 → bit 4 and bit 5
missingSensors = (status AND 0x0F) XOR 0x0F	0x0F XOR 0x0F = 0x00 (not reached)	0x0B XOR 0x0F = 0x04 → bit 2
Output	FLIGHT READY (radio link bit 6 is set but ignored)	FAULT: Bit 4 – Low Battery; FAULT: Bit 5 – Over-Temperature; MISSING: Bit 2 – Barometric altimeter; FLIGHT NOT READY

Note: the question describes 0x3B as "low battery with missing sensor", but 0x3B (0011 1011) also has bit 5 (over-temperature, 0x20) set, so the routine correctly reports both faults.

Question 4: Optimized Prime Factorization & Divisor Engine

Topic: Control Structures (Iteration: for/while, Jump: break, continue, goto)

Design Overview

- **Input validation** uses a GOTO jump back to a label until N > 0.
- **N = 1** is handled explicitly: it has no proper divisors (sum 0), so it is Deficient.
- **O(√N) search:** divisors come in pairs (d, N/d); only d from 2 to ⌊√N⌋ needs testing. The sum starts at 1 (the divisor 1 is always proper).
- **CONTINUE** skips non-divisors; when d = N/d (perfect square) the partner is added only once.
- **Classification** by comparing sumOfDivisors with N. As a bonus, N > 1 with sum = 1 means N is prime.

Task 1 – Structured pseudo-code

Structured pseudo-code

```
BEGIN DivisorAndPrimeEngine
  INTEGER N, d, limit, pair, sumOfDivisors

  ReadInput:
  DISPLAY "Enter a positive integer N: "
  READ N
  IF N <= 0 THEN
    DISPLAY "Error: N must be greater than 0. Try again."
    GOTO ReadInput
  ENDIF

  IF N = 1 THEN
    DISPLAY "1 has no proper divisors (sum = 0): Deficient"
    GOTO Finish
  ENDIF

  sumOfDivisors <- 1
  limit <- FLOOR(SQRT(N))

  FOR d <- 2 TO limit STEP 1
    IF N MOD d <> 0 THEN
      CONTINUE
    ENDIF
    pair <- N DIV d
    sumOfDivisors <- sumOfDivisors + d
    IF pair <> d THEN
      sumOfDivisors <- sumOfDivisors + pair
    ENDIF
  ENDFOR

  DISPLAY "Sum of proper divisors = ", sumOfDivisors
  IF sumOfDivisors = N THEN
    DISPLAY N, " is a Perfect Number"
  ELSE
    IF sumOfDivisors > N THEN
      DISPLAY N, " is an Abundant Number"
    ELSE
      DISPLAY N, " is a Deficient Number"
      IF sumOfDivisors = 1 THEN
        DISPLAY N, " is also a Prime Number"
      ENDIF
```

```
    ENDIF
  ENDIF

  Finish:
  DISPLAY "Done"
END
```

Task 2 – Trace tables

N = 28 (limit = ⌊√28⌋ = 5; sumOfDivisors starts at 1)

d	N MOD d	N / d (pair)	Action	sumOfDivisors
—	—	—	initialize	1
2	0	14	divides; add 2 and 14	1 + 2 + 14 = 17
3	1	—	CONTINUE	17
4	0	7	divides; add 4 and 7	17 + 4 + 7 = 28
5	3	—	CONTINUE	28

Sum = 28 = N → **Perfect Number**.

N = 12 (limit = ⌊√12⌋ = 3; sumOfDivisors starts at 1)

d	N MOD d	N / d (pair)	Action	sumOfDivisors
—	—	—	initialize	1
2	0	6	divides; add 2 and 6	1 + 2 + 6 = 9
3	0	4	divides; add 3 and 4	9 + 3 + 4 = 16

Sum = 16 > 12 → **Abundant Number**.

Question 5: State-Machine ATM Controller with Sentinel Loops

Topic: Control Structures (switch-case, while, nested if-else), Data Types

Design Overview

- **Authentication phase:** a counter-controlled WHILE loop gives at most 3 PIN attempts and stops early on success. After 3 failures the card is locked and the program ends without entering the menu.
- **Session phase:** a sentinel-controlled WHILE loop repeats the menu until the user chooses option 4 (Exit).
- **SWITCH-CASE** dispatches the menu options; DEFAULT handles invalid choices.
- **Withdrawal** uses nested IF: amount > 0, then amount ≤ balance, then `amount MOD 100 = 0`.
- **Assumption:** the opening balance is $1,000.00 for the demonstration.

Task 1 – Structured pseudo-code

Structured pseudo-code

```
BEGIN ATM_SessionController
  CONSTANT INTEGER CORRECT_PIN  <- 4821
  CONSTANT INTEGER MAX_ATTEMPTS <- 3
  INTEGER enteredPin, attempts, choice
  REAL    balance, amount
  BOOLEAN authenticated, sessionActive

  balance       <- 1000.00
  attempts      <- 0
  authenticated <- FALSE

  // ---- Authentication phase ----
  WHILE attempts < MAX_ATTEMPTS AND authenticated = FALSE DO
    DISPLAY "Enter 4-digit PIN: "
    READ enteredPin
    attempts <- attempts + 1
    IF enteredPin = CORRECT_PIN THEN
      authenticated <- TRUE
    ELSE
      IF attempts < MAX_ATTEMPTS THEN
        DISPLAY "Incorrect PIN. Attempts remaining: ",
                MAX_ATTEMPTS - attempts
      ENDIF
    ENDIF
  ENDWHILE

  IF authenticated = FALSE THEN
    DISPLAY "Card locked after 3 failed attempts."
  ELSE
    // ---- Session phase (sentinel = option 4) ----
    sessionActive <- TRUE
    WHILE sessionActive = TRUE DO
      DISPLAY "1: Balance  2: Deposit  3: Withdraw  4: Exit"
      READ choice
      SWITCH choice
        CASE 1:
          DISPLAY "Available balance: $", balance
        CASE 2:
          DISPLAY "Enter deposit amount: "
          READ amount
```

```
          IF amount > 0 THEN
            balance <- balance + amount
            DISPLAY "Deposited. New balance: $", balance
          ELSE
            DISPLAY "Invalid amount. Must be positive."
          ENDIF
        CASE 3:
          DISPLAY "Enter withdrawal amount: "
          READ amount
          IF amount <= 0 THEN
            DISPLAY "Invalid amount. Must be positive."
          ELSE
            IF amount > balance THEN
              DISPLAY "Insufficient funds."
            ELSE
              IF amount MOD 100 <> 0 THEN
                DISPLAY "Amount must be a multiple of 100."
              ELSE
                balance <- balance - amount
                DISPLAY "Withdrawn. New balance: $", balance
              ENDIF
            ENDIF
          ENDIF
        CASE 4:
          sessionActive <- FALSE
          DISPLAY "Session ended. Please take your card."
        DEFAULT:
          DISPLAY "Invalid option. Choose 1 to 4."
      ENDSWITCH
    ENDWHILE
  ENDIF
END
```

Task 2 – Dry-run trace (opening balance $1,000.00)

Step	Input	Checks	State	Output
1	—	attempts = 0, authenticated = FALSE; loop test 0 < 3 is true	balance 1000	prompt for PIN
2	PIN 1234	attempts = 1; 1234 ≠ 4821; 1 < 3	attempts 1	"Incorrect PIN. Attempts remaining: 2"
3	PIN 4821	attempts = 2; 4821 = 4821 → authenticated = TRUE	attempts 2	loop ends; menu shown
4	Choice 3, amount 250	250 > 0 ✓; 250 ≤ 1000 ✓; 250 MOD 100 = 50 ≠ 0 ✗	balance 1000	"Amount must be a multiple of 100."
5	Choice 3, amount 300	300 > 0 ✓; 300 ≤ 1000 ✓; 300 MOD 100 = 0 ✓	balance 700	"Withdrawn. New balance: $700.00"
6	Choice 4	sessionActive = FALSE; loop test fails	balance 700	"Session ended. Please take your card."

Lock-out case: three wrong PINs give attempts = 3, the loop test 3 < 3 fails, authenticated is still FALSE, "Card locked after 3 failed attempts." is shown and the program ends without showing the menu.